BLIND SPOT DETECTION SYSTEM

USING ULTRASONIC SENSORS & OLED DISPLAY

TABLE OF CONTENTS

1. Abstract

A real-time blind spot monitoring system using dual ultrasonic sensors (HC-SR04) to detect nearby obstacles and display distance on an OLED (I2C) screen. The system also triggers visual (LED) and audible (buzzer) alerts when an object enters the predefined threshold zone.

2. Problem Statement

Blind spots in vehicles are a major safety hazard, especially during lane changes. Traditional mirrors cannot cover certain rear angles, leading to undetected nearby vehicles. This system attempts to solve that by actively monitoring both sides and providing clear alerts.

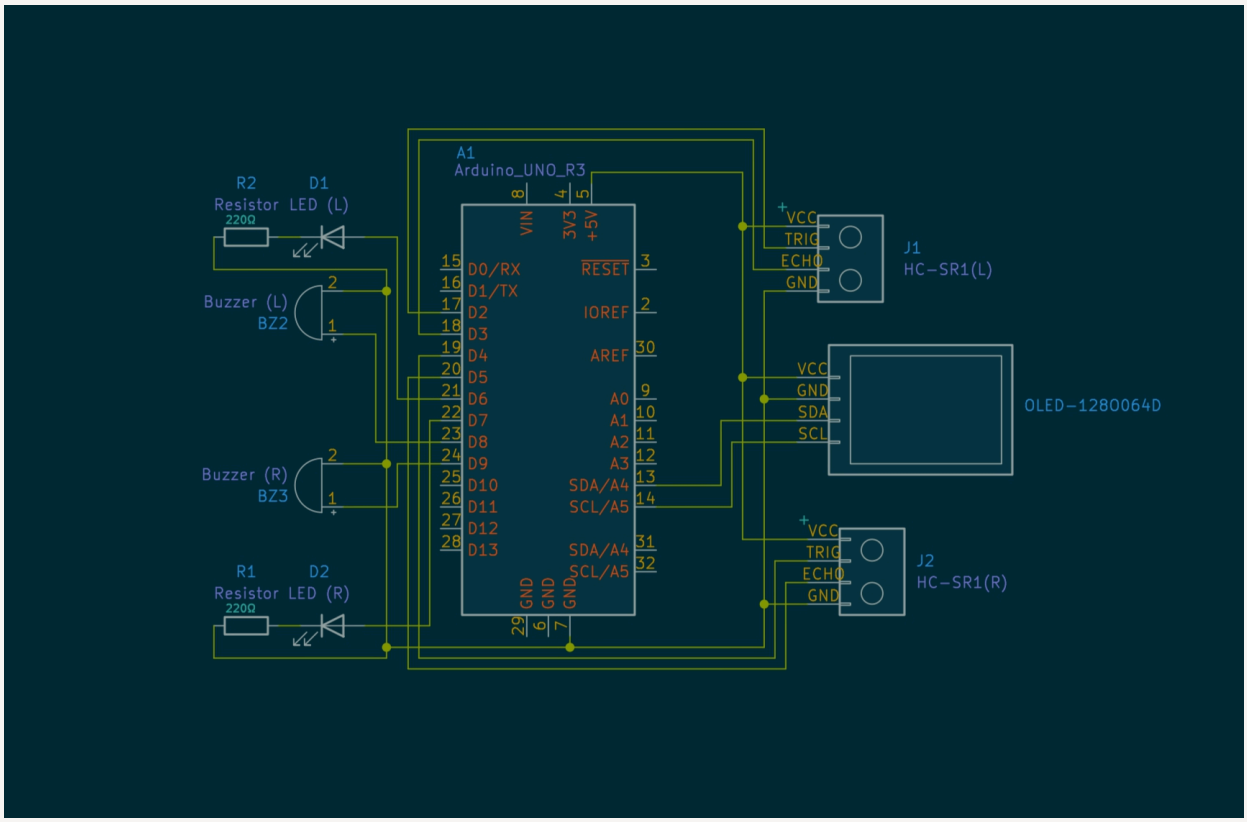
3. Tools & Technologies

- Arduino Uno R3
- HC-SR04 ultrasonic sensors
- OLED display (SSD1306 over I2C)
- LEDs, Buzzers, Resistors
- Wokwi simulator
- KiCad (schematic)
- Arduino IDE

4. Bill of Materials (BoM)

Qty	Component	Description
1	Arduino Uno R3	Microcontroller
2	HC-SR04	Ultrasonic Distance Sensor
1	OLED Display	SSD1306, I2C, 0.96", 4-pin
2	LED (Red)	Visual alert
2	Resistor (220Ω)	For LEDs
2	Buzzer (5V Piezo)	Audible alert
-	Jumper Wires	All connections
-	Breadboard	Optional, for real prototype

5. Schematic Diagram



6. Component Justification

- **Arduino Uno R3**

  Chosen for its simplicity, availability, and support. It provides enough digital I/O pins for interfacing with two ultrasonic sensors, two LEDs, buzzers, and an I2C OLED. It's beginner-friendly and industry-accepted for small embedded prototypes.

- **HC-SR04 Ultrasonic Sensor (×2)**

  Used to detect objects in left and right blind spots. It's inexpensive, has a usable range of 2 cm to 400 cm, and operates with a straightforward trigger/echo mechanism. Provides reliable distance data for obstacle detection without needing a camera or image processing.

- **OLED Display (SSD1306, I2C)**

  Chosen over LCD due to its compact size, low power consumption, and high contrast. It communicates via I2C, reducing wiring complexity. The SSD1306 library is mature and supports live data updates with easy formatting.

- **LEDs (×2)**

  Used as visual alerts when an object is detected in the blind spot. Each LED corresponds to one side (left/right). They provide immediate, simple feedback for the user.

- **Resistors (220Ω ×2)**

  Used to limit current to the LEDs and protect them from damage. 220Ω is a standard value suitable for 5V systems.

- **Buzzers (×2)**

  Added to provide audible alerts alongside visual LEDs. Especially useful in noisy environments or when the display is not in direct view.

- **Wokwi Simulator**

  Used for circuit simulation, wiring validation, and live sensor testing. Supports virtual OLEDs and sensors, enabling accurate simulation without hardware.

- **KiCad**

  Used for creating professional-quality schematics suitable for documentation and portfolio presentation. Preferred over Fritzing due to open-source nature and industry relevance.

7. Arduino Code & Explanation

- **Setup Section**

```
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Wire.h>
```

  - These libraries handle I2C communication and display rendering.
  - Wire.h is for I2C bus.
  - Adafruit_GFX + SSD1306 controls the OLED display.

- **OLED Display Setup**

```
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```

  - Creates a display object.
  - Width & height are standard for 0.96" OLED.
  - -1 means no reset pin is used (handled internally in Wokwi).

- **Pin Declarations**

```
const int trigLeft = 2, echoLeft = 3;
const int trigRight = 4, echoRight = 5;
const int ledLeft = 6, ledRight = 7;
const int buzzerLeft = 8, buzzerRight = 9;
const int threshold = 30;
```

  - GPIO pin mappings for each sensor and output device.
  - threshold sets the distance (in cm) to trigger alerts.

- **(setup) Function**

```
void setup() {
  Serial.begin(9600);
  pinMode(...) // all pin setups
  display.begin(...);
}
```

  - Initializes serial monitor for debugging.
  - Sets pin modes for sensors, LEDs, buzzers.
  - Starts OLED using I2C address 0x3C.

- **Distance Measuring Function**

```
long getDistanceCM(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW); delayMicroseconds(2);
  digitalWrite(trigPin, HIGH); delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duration = pulseIn(echoPin, HIGH);
  return duration * 0.034 / 2;
}
```

  - Triggers ultrasonic pulse
  - Reads echo time
  - Converts to distance (speed of sound = 0.034 cm/µs)

- **loop() Function – Main Logic**

```
void loop() {
  long leftDistance = getDistanceCM(...);
  long rightDistance = getDistanceCM(...);
```

  - Calls the distance function for both sensors.

```
  digitalWrite(ledLeft, (leftDistance <= threshold) ? HIGH : LOW);
  digitalWrite(buzzerLeft, (leftDistance <= threshold) ? HIGH : LOW);
```

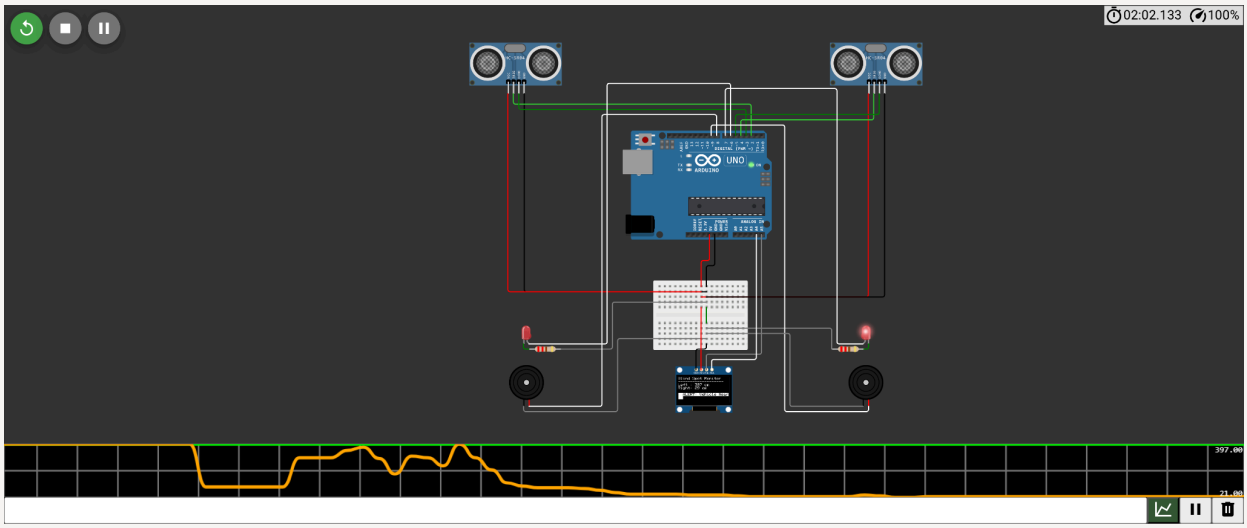
  - Turns LED and buzzer ON if object detected.

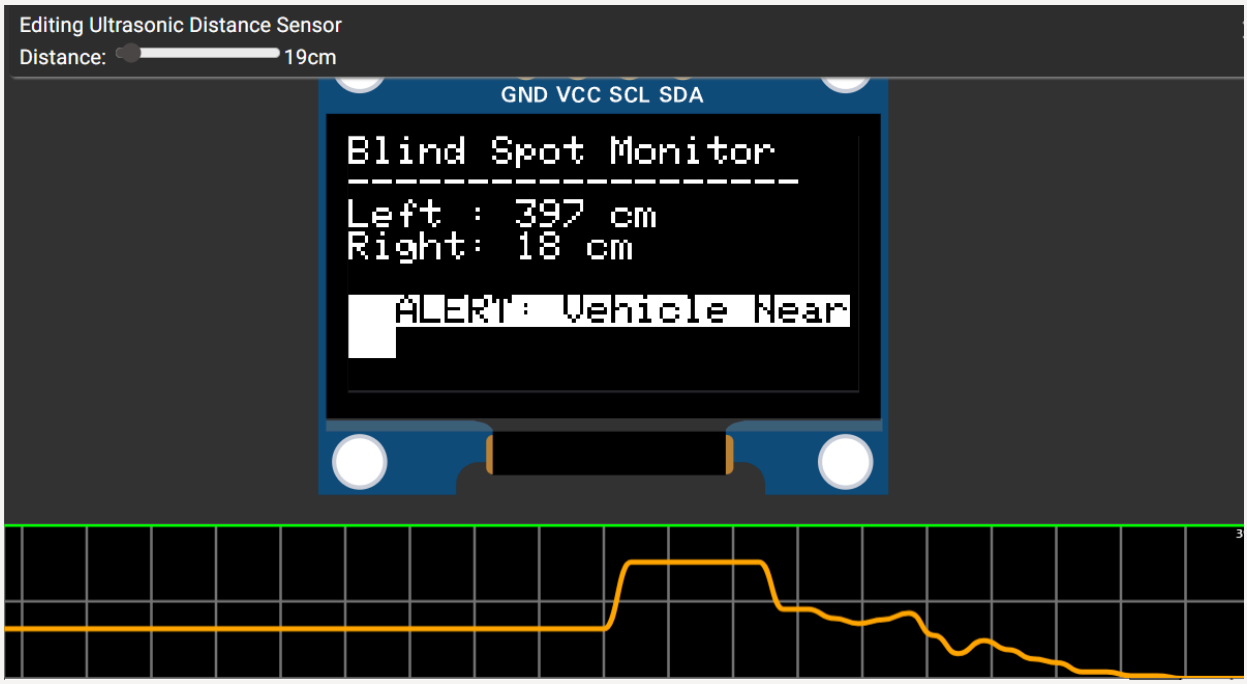
```
display.clearDisplay();
display.setCursor(0,0);
display.print("Blind Spot Monitor");
...
```

- Displays formatted text on the OLED.
- Shows left and right distances.
- If object is close, shows “ALERT: Vehicle Near”

This code loop runs every ~300 ms and updates distance + alert outputs live.

8. Output Screenshots





9. Challenges Faced & Learnings

- Learned to interface multiple peripherals (I2C + Ultrasonic)
- Understood I2C protocol (SDA/SCL wiring, display addressing)
- First exposure to schematic creation in KiCad
- Recovered from delays and finished a portfolio-ready project

10. Future Scope

- Add Bluetooth module to send alerts to phone or helmet HUD
- Integrate with CAN bus or vehicle system for in-car displays
- Add adaptive threshold based on speed or light conditions
- Expand to rear and front proximity monitoring

11. Links

GitHub:
https://github.com/DannyJusvin/Blind-Spot-Detection-System

Wokwi Simulation: https://wokwi.com/projects/437343000411654145

Schematic Diagram: BlindSpot_Schematics.pdf

Project Logs: Project Logs.pdf